How Attention Parameters Are Trained

The Main Question

In attention mechanisms, we compute attention weights:

$\alpha_1, \alpha_2, \alpha_3, \ldots$

These weights determine:

- which words to focus on
- how much importance to give each input token

A natural question arises:

> If we do not have true labels for attention weights, then how are attention parameters trained?

Core Idea

Attention weights are NOT directly supervised.

We do not provide labels such as:

- “focus on this word”
- “look here”
- “give 80% attention to this token”

Instead:

Attention is learned indirectly through the final task loss.

The model automatically discovers useful attention patterns during training.

High-Level Intuition

Suppose the task is machine translation.

Input:

“The cat sat”

Output:

“Le chat assis”

While generating the word:

“chat”

The model should focus mostly on:

“cat”

But nobody explicitly tells the model this.

The model learns it automatically because focusing on “cat” reduces prediction error.

Step-by-Step Training Flow

Step 1: Encoder Creates Hidden States

Input sentence:

"The" "cat" "sat"

The encoder converts each word into hidden representations:

h1, h2, h3

Each hidden state contains contextual information about a word.

Step 2: Decoder Computes Attention Scores

At decoder step t:

The decoder compares its current state st with every encoder hidden state hi.

A scoring function computes:

$e_i = score(s_t, h_i)$

These are raw importance scores.

Step 3: Softmax Produces Attention Weights

The scores are normalized using softmax:

$\alpha_i = \exp(e_i) / \Sigma \exp(e_j)$

Properties:

- all α_i are positive
- all α_i sum to 1
- α_i behave like probabilities

Example:

Word	Attention Weight
The	0.1
cat	0.8
sat	0.1

The decoder focuses mostly on “cat”.

Step 4: Context Vector Computation

The context vector is computed as a weighted sum:

$c_t = \Sigma \alpha_i h_i$

The attention weights decide:

- which hidden states contribute more
- which words are more important

This context vector is passed to the decoder.

Step 5: Decoder Predicts Output

Using:

- previous words
- decoder state
- context vector

The decoder predicts the next word.

Suppose the model predicts:

“chien” (dog)

instead of:

“chat” (cat)

Then prediction error becomes large.

Step 6: Loss Computation

A loss function measures prediction error.

Example:

- cross entropy loss

Higher error means:

- larger loss

Step 7: Backpropagation

Gradients now flow backward through:

- decoder
- context vector
- attention weights
- scoring function
- encoder

The model learns:

> “To reduce prediction error, attention should focus more on the correct word.”

Thus:

- attention parameters are automatically updated
- useful attention patterns gradually emerge

Important Insight

Attention is NOT supervised directly.

Instead:

Attention is self-organized through optimization.

The network learns where to focus because:

- focusing correctly reduces loss

Analogy

Imagine a student solving questions from a textbook.

Nobody explicitly says:

> “Look at line 7.”

But after repeatedly making mistakes, the student learns:

- which parts are important
- where attention should go

Attention mechanisms learn similarly.

Mathematical View

In transformers, attention depends on trainable matrices.

Input representations:

X

Trainable matrices:

WQ, WK, WV

The model computes:

Q = XWQ K = XWK V = XWV

where:

- Q = Queries
- K = Keys
- V = Values

Attention scores are computed using:

Attention(Q,K,V) = softmax(QKT / $\sqrt{}$d) V

The matrices:

- WQ
- WK
- WV

are trainable parameters.

These parameters are updated using:

- gradient descent
- backpropagation
- final task loss

No Ground-Truth Attention Labels Needed

This is a very important concept.

Most deep learning models do not have supervision for intermediate computations.

We usually supervise only:

- final outputs

The network internally learns:

- useful representations
- useful attention patterns
- meaningful feature importance

Example in Biomolecule Project

Suppose:

Input:

- molecular graph

Output:

- toxicity prediction

The attention mechanism may automatically learn:

- which atoms matter most
- which bonds are important
- which functional groups influence toxicity

No scientist manually provides attention labels.

The model discovers them automatically because:

- attending to important regions reduces prediction error.

Key Takeaways

1. Attention weights are not directly supervised.
2. We do not provide true attention labels.
3. Attention parameters are learned through backpropagation.
4. Final task loss drives attention learning.
5. The model automatically discovers where to focus.
6. Attention is a differentiable soft-selection mechanism.

Final Intuition

Attention learns:

> “What parts of the input are most useful for solving the task?”

And it learns this entirely from:

- prediction success
- optimization
- gradient descent